Choosing a hosting provider

Except for the most basic recommendations, the answer to *What should I do?* is always going to be *It depends*. I have been using Linux since Slackware 3.2 (that's the late 90s), so a VPS is fairly comfortable for me. If you're familiar with the Unix world and have installed software like web servers and databases before, the VPS will likely be a good fit. For my smaller clients, I usually start with OpalStack; it gives full shell access and control, but they take care of the database and web server instance, so it is a nice middle ground.

At scale, you'll want more control over how many servers you're running, how they're load balanced, and where background tasks operate. Within the VPS world, this means more work, but you can tune it to your heart's content.

PaaS services tend to be a bit more expensive. Heroku used to be the go-to because it was free, but that ended in November 2022. That isn't a reason not to use their platform; all the other PaaS providers charge as well. And you get what you pay for: full-time professionals take care of the servers, infrastructure, and rollout scripts for you.

I personally don't use PaaS for my clients, but I have worked with folks who do, and they swear by it. At scale, PaaS can become expensive, but if you're serving a lot of users, automatically adding more instances to keep the site running smoothly is a big win.

B.3 Synchronous vs. asynchronous

WSGI is the Python standard interface for interacting with a web server, and ASGI is its asynchronous equivalent. When you interact with a server synchronously, it doesn't return a result to you until the result is ready. With a web page, that means getting the HTML all in one go. Synchronous systems tend to be pull based, meaning it is up to the client (the web browser) to ask the server for something.

Asynchronous systems tend to be bidirectional. You might ask for something and get a response, or the system might send you something on its own. Chat typically gets built using asynchronous technology: the server notifies you when you receive a message. To implement chat in a synchronous server, your client has to ask the server *Is anything new for me?* with some regularity. This is known as *polling* the server.

In the last few years, there has been a trend towards asynchronous based web frameworks. Django originally was a synchronous-only tool kit, but asynchronous features have been introduced over time. Daphne, Uvicorn, and Hypercorn are all asynchronous web servers that can interact with Django over ASGI.

For a "normal" Django project, switching to using Daphne and the like is as simple as pointing Daphne at asgi.py instead of wsgi.py in your project configuration folder. Typically, though, the reason for going asynchronous is to take advantage of the features it enables. If you want to build a chat server using Django, you're going to need extra tools. The django-channels third-party library integrates WebSockets with Django, allowing full bidirectional communication with your web server over JavaScript, and as it is Django based, it uses views to do this.